

EasyNotes v2

Plan de Desarrollo de Rutas API

Sistema de Gestion Academica

Julio 2026

Persona	Rol	Modulo
Yorman	Infraestructura y Setup	Base, Auth, Usuarios, Bitacora
Sneider	Modulo Institucional	Institucion, Sede, Nucleo, Catalogo
Santiago	Modulo Academico	Anio, Area, Asignatura, Grupo, Carga
Martin	Matricula y Notas	Prematricula, Matricula, Indicador, Calif
Avila	Modulos Transversales	Comunicados, Observador, Pagos, Elecciones

Tabla de Contenido

1. Convenciones generales
2. Yorman - Infraestructura y Setup
3. Sneider - Modulo Institucional
4. Santiago - Modulo Academico
5. Martin - Matricula y Calificaciones
6. Avila - Modulos Transversales
7. Dependencias y orden de trabajo
8. Anexos: Codigos base

1. Convenciones Generales

1.1 Formato de respuesta unificada

```
{
  "ok": true/false,
  "data": ...,
  "message": "...",
  "error": "..."
}
```

1.2 Ruta base por recurso

Recurso	Ruta	Responsable
Auth	/api/auth	Yorman
Usuarios	/api/usuarios	Yorman
Bitacora	/api/bitacora	Yorman
Instituciones	/api/instituciones	Sneider
Sedes	/api/sedes	Sneider
Nucleos	/api/nucleos	Sneider
Solicitudes Registro	/api/solicitudes-registro	Sneider
Catalogos	/api/catalogos	Sneider
Anios Academicos	/api/anios-academicos	Santiago
Areas	/api/areas	Santiago
Asignaturas	/api/asignaturas	Santiago
Grupos	/api/grupos	Santiago
Carga Academica	/api/carga-academica	Santiago
Prematriculas	/api/prematriculas	Martin
Matriculas	/api/matriculas	Martin
Indicadores	/api/indicadores	Martin
Actividades	/api/actividades	Martin
Calificaciones	/api/calificaciones	Martin
Comunicados	/api/comunicados	Avila
Observador	/api/observador	Avila
Excusas	/api/excusas	Avila
Conceptos Contables	/api/conceptos-contables	Avila
Pagos	/api/pagos	Avila
Elecciones	/api/elecciones	Avila
Eventos Electorales	/api/eventos-electorales	Avila
Votos	/api/votos	Avila

1.3 Plantilla de controller CRUD

```
const Model = require('../models/Modelo');

const getAll = async (req, res) => {
  try {
    const filter = req.usuario?.institucionId
      ? { institucionId: req.usuario.institucionId }
      : {};
    const data = await Model.find(filter);
    res.json({ ok: true, data, message: 'Listado obtenido' });
  } catch (error) {
    res.status(500).json({ ok: false, message: 'Error al listar',
      error: error.message });
  }
};

const getById = async (req, res) => {
  try {
    const data = await Model.findById(req.params.id);
```

```
    if (!data) return res.status(404)
    .json({ ok: false, message: 'No encontrado' });
    res.json({ ok: true, data });
  } catch (error) {
    res.status(500).json({ ok: false, message: 'Error al obtener',
      error: error.message });
  }
};

const create = async (req, res) => {
  try {
    const body = { ...req.body };
    if (req.usuario?.institucionId)
      body.institucionId = req.usuario.institucionId;
    const data = await Model.create(body);
    res.status(201).json({ ok: true, data,
      message: 'Creado correctamente' });
  } catch (error) {
    res.status(400).json({ ok: false, message: 'Error al crear',
      error: error.message });
  }
};

const update = async (req, res) => {
  try {
    const data = await Model.findByIdAndUpdate(
      req.params.id, req.body,
      { new: true, runValidators: true });
    if (!data) return res.status(404)
      .json({ ok: false, message: 'No encontrado' });
    res.json({ ok: true, data,
      message: 'Actualizado correctamente' });
  } catch (error) {
    res.status(400).json({ ok: false, message: 'Error al actualizar',
      error: error.message });
  }
};

const remove = async (req, res) => {
  try {
    const data = await Model.findByIdAndDelete(req.params.id);
    if (!data) return res.status(404)
      .json({ ok: false, message: 'No encontrado' });
    res.json({ ok: true, message: 'Eliminado correctamente' });
  } catch (error) {
    res.status(500).json({ ok: false, message: 'Error al eliminar',
      error: error.message });
  }
};

module.exports = { getAll, getById, create, update, remove };
```

2. Yorman - Infraestructura y Setup

Dependencia: Nadie. Todos los demas dependen de el.

Archivos a crear

Archivo	Contenido
src/config/constants.js	ROLES, ESTADOS, TIPOS_DOCUMENTO, GENEROS
src/middleware/auth.js	protect, authorize, generarToken
src/routes/index.js	Centralizador de todas las rutas
server.js (reescribir)	Express + middleware + rutas
controllers/auth.controller.js	login, me, cambiarPassword
routes/auth.routes.js	Rutas de autenticacion
controllers/usuario.controller.js	CRUD + endpoints extra
routes/usuario.routes.js	Rutas de usuario
controllers/bitacora.controller.js	Solo lectura
routes/bitacora.routes.js	Rutas de bitacora

Endpoints de Auth

Metodo	Ruta	Funcion	Auth
POST	/api/auth/login	login	No
GET	/api/auth/me	me	Si (protect)
PUT	/api/auth/password	cambiarPassword	Si (protect)

Endpoints de Usuario

Metodo	Ruta	Funcion
GET	/api/usuarios	getAll
GET	/api/usuarios/:id	getById
POST	/api/usuarios	create
PUT	/api/usuarios/:id	update
DELETE	/api/usuarios/:id	remove
GET	/api/usuarios/:id/estudiantes	misEstudiantes
GET	/api/usuarios/:id/acudientes	misAcudientes
GET	/api/usuarios/buscar/:documento	buscarPorDocumento

Endpoints de Bitacora

Metodo	Ruta	Funcion
GET	/api/bitacora	getAll
GET	/api/bitacora/:id	getById
GET	/api/bitacora/usuario/:usuarioid	getByUsuario

3. Sneider - Modulo Institucional

Dependencia: constants.js (Yorman), auth.js (Yorman), routes/index.js (Yorman)

Modelos a implementar

Controller + Routes	Modelo	Notas
institucion	Institucion	Configuracion con defaults del schema
sede	Sede	Filtrar por institucionId
direccionNucleo	DireccionNucleo	No tiene institucionId
solicitudRegistro	SolicitudRegistro	Flujo pendiente -> aprobada/rechazada
catalogo	Catalogo	CRUD generico por tipo e institucionId

Endpoints de Institucion

Metodo	Ruta	Funcion
GET	/api/instituciones	getAll
GET	/api/instituciones/:id	getById
POST	/api/instituciones	create
PUT	/api/instituciones/:id	update
DELETE	/api/instituciones/:id	remove

Endpoints de Sede

Metodo	Ruta	Funcion
GET	/api/sedes	getAll
GET	/api/sedes/:id	getById
POST	/api/sedes	create
PUT	/api/sedes/:id	update
DELETE	/api/sedes/:id	remove

Endpoints de Nucleo

Metodo	Ruta	Funcion
GET	/api/nucleos	getAll
GET	/api/nucleos/:id	getById
POST	/api/nucleos	create
PUT	/api/nucleos/:id	update
DELETE	/api/nucleos/:id	remove

Endpoints de Solicitudes Registro

Metodo	Ruta	Funcion
GET	/api/solicitudes-registro	getAll
GET	/api/solicitudes-registro/:id	getById
POST	/api/solicitudes-registro	create
PUT	/api/solicitudes-registro/:id	update
DELETE	/api/solicitudes-registro/:id	remove
PUT	/api/solicitudes-registro/:id/aprobar	aprobar (crea Institucion)
PUT	/api/solicitudes-registro/:id/rechazar	rechazar

Endpoints de Catalogo

Metodo	Ruta	Funcion
GET	/api/catalogos	getAll
GET	/api/catalogos/:id	getById

POST	/api/catalogos	create
PUT	/api/catalogos/:id	update
DELETE	/api/catalogos/:id	remove
GET	/api/catalogos/tipo/:tipo	getByTipo

4. Santiago - Modulo Academico

Dependencia: constants.js (Yorman), auth.js (Yorman), routes/index.js (Yorman)

Modelos a implementar

Controller + Routes	Modelo	Notas
anioAcademico	AnioAcademico	Solo un ano activo por institucion
area	Area	CRUD simple
asignatura	Asignatura	Filtrar por areald
grupo	Grupo	Filtrar por anioAcademicold
cargaAcademica	CargaAcademica	Validar indice unico

Endpoints

Metodo	Ruta	Funcion
GET	/api/anios-academicos	getAll
GET	/api/anios-academicos/:id	getById
POST	/api/anios-academicos	create
PUT	/api/anios-academicos/:id	update
DELETE	/api/anios-academicos/:id	remove
PUT	/api/anios-academicos/:id/activar	activar (desactiva otros)
PUT	/api/anios-academicos/:id/cerrar	cerrar
GET	/api/areas	getAll
GET	/api/areas/:id	getById
POST	/api/areas	create
PUT	/api/areas/:id	update
DELETE	/api/areas/:id	remove
GET	/api/asignaturas	getAll
GET	/api/asignaturas/:id	getById
POST	/api/asignaturas	create
PUT	/api/asignaturas/:id	update
DELETE	/api/asignaturas/:id	remove
GET	/api/asignaturas/area/:areald	getByArea
GET	/api/grupos	getAll
GET	/api/grupos/:id	getById
POST	/api/grupos	create
PUT	/api/grupos/:id	update
DELETE	/api/grupos/:id	remove
GET	/api/grupos/anio/:anioAcademicold	getByAnio
GET	/api/carga-academica	getAll
GET	/api/carga-academica/:id	getById
POST	/api/carga-academica	create
PUT	/api/carga-academica/:id	update
DELETE	/api/carga-academica/:id	remove
GET	/api/carga-academica/docente/:docenteld	getByDocente
GET	/api/carga-academica/grupo/:grupold	getByGrupo

5. Martin - Matricula y Calificaciones

Dependencia: constants.js (Yorman), auth.js (Yorman), routes/index.js (Yorman)

Flujo critico del sistema. Endpoint masivo de notas obligatorio.

Modelos a implementar

Controller + Routes	Modelo	Notas
prematricula	Prematricula	Al aprobar crea Usuario + Matricula
matricula	Matricula	retirar, promover
indicador	Indicador	Filtrar por asignaturald + periodo
actividad	Actividad	Filtrar por grupold + periodo
calificacion	Calificacion	POST /masivo para notas de todo un grupo

Endpoints de Prematricula

Metodo	Ruta	Funcion
GET	/api/prematriculas	getAll
GET	/api/prematriculas/:id	getById
POST	/api/prematriculas	create
PUT	/api/prematriculas/:id	update
DELETE	/api/prematriculas/:id	remove
PUT	/api/prematriculas/:id/aprobar	aprobar
PUT	/api/prematriculas/:id/rechazar	rechazar

Endpoints de Matricula

Metodo	Ruta	Funcion
GET	/api/matriculas	getAll
GET	/api/matriculas/:id	getById
POST	/api/matriculas	create
PUT	/api/matriculas/:id	update
DELETE	/api/matriculas/:id	remove
PUT	/api/matriculas/:id/retirar	retirar
PUT	/api/matriculas/:id/promover	promover
GET	/api/matriculas/grupo/:grupold	getByGrupo

Endpoints de Indicador

Metodo	Ruta	Funcion
GET	/api/indicadores	getAll
GET	/api/indicadores/:id	getById
POST	/api/indicadores	create
PUT	/api/indicadores/:id	update
DELETE	/api/indicadores/:id	remove
GET	/api/indicadores/asignatura/:id/periodo/:p	getByAsignaturaPeriodo

Endpoints de Actividad

Metodo	Ruta	Funcion
GET	/api/actividades	getAll
GET	/api/actividades/:id	getById
POST	/api/actividades	create
PUT	/api/actividades/:id	update
DELETE	/api/actividades/:id	remove

GET /api/actividades/grupo/:id/periodo/:p

getByGrupoPeriodo

Endpoints de Calificacion (Martin)

Metodo	Ruta	Funcion
GET	/api/calificaciones	getAll
GET	/api/calificaciones/:id	getById
POST	/api/calificaciones	create
PUT	/api/calificaciones/:id	update
DELETE	/api/calificaciones/:id	remove
POST	/api/calificaciones/masivo	guardarNotas (array)
GET	/api/calificaciones/grupo/:g/asig/:a/per/:p	getByGrupoAsigPeriodo
GET	/api/calificaciones/estudiante/:e/aniol/:a	getBoletin

6. Avila - Modulos Transversales

Dependencia: constants.js (Yorman), auth.js (Yorman), routes/index.js (Yorman)

Modelos a implementar

Controller + Routes	Modelo	Notas
comunicados	Comunicados	CRUD + marcarLeido
observador	Observador	CRUD + agregar seguimiento
excusas	Excusas	CRUD + aprobar/rechazar
conceptosContables	ConceptosContables	CRUD simple
pagos	Pagos	CRUD + registrarPago
elecciones	Elecciones	CRUD + votar, resultados, abrir/cerrar
eventoElectoral	EventoElectoral	CRUD simple
voto	Voto	CRUD + filtrar por evento/candidato

Endpoints

Metodo	Ruta	Funcion
GET	/api/comunicados	getAll
GET	/api/comunicados/:id	getById
POST	/api/comunicados	create
PUT	/api/comunicados/:id	update
DELETE	/api/comunicados/:id	remove
PUT	/api/comunicados/:id/leer	marcarLeido
GET	/api/observador	getAll
GET	/api/observador/:id	getById
POST	/api/observador	create
PUT	/api/observador/:id	update
DELETE	/api/observador/:id	remove
POST	/api/observador/:id/seguimiento	agregarSeguimiento
GET	/api/observador/estudiante/:id	getByEstudiante
GET	/api/excusas	getAll
GET	/api/excusas/:id	getById
POST	/api/excusas	create
PUT	/api/excusas/:id	update
PUT	/api/excusas/:id/aprobar	aprobar
PUT	/api/excusas/:id/rechazar	rechazar
GET	/api/conceptos-contables	getAll
GET	/api/conceptos-contables/:id	getById
POST	/api/conceptos-contables	create
PUT	/api/conceptos-contables/:id	update
DELETE	/api/conceptos-contables/:id	remove
GET	/api/pagos	getAll
GET	/api/pagos/:id	getById
POST	/api/pagos	create
PUT	/api/pagos/:id	update
DELETE	/api/pagos/:id	remove

PUT	/api/pagos/:id/regar-pago	regarPago
GET	/api/pagos/estudiante/:id	getByEstudiante
GET	/api/ecciones	getAll
GET	/api/ecciones/:id	getById
POST	/api/ecciones	create
PUT	/api/ecciones/:id	update
DELETE	/api/ecciones/:id	remove
POST	/api/ecciones/:id/votar	votar
GET	/api/ecciones/:id/resultados	resultados
PUT	/api/ecciones/:id/abrir	abrir
PUT	/api/ecciones/:id/cerrar	cerrar
GET	/api/eventos-electorales	getAll
GET	/api/eventos-electorales/:id	getById
POST	/api/eventos-electorales	create
PUT	/api/eventos-electorales/:id	update
DELETE	/api/eventos-electorales/:id	remove
GET	/api/votos	getAll
GET	/api/votos/:id	getById
POST	/api/votos	create
DELETE	/api/votos/:id	remove
GET	/api/votos/evento/:eventid	getByEvento
GET	/api/votos/candidato/:candidatid	getByCandidato

7. Dependencias y Orden de Trabajo

Diagrama de dependencias

Yorman (constants, server, auth)
 +-- Sneider (usa constants + auth)
 +-- Santiago (usa constants + auth)
 +-- Martin (usa constants + auth)
 +-- Avila (usa constants + auth)

Orden recomendado

Paso	Quien	Que entrega
1	Yorman	constants.js y middleware/auth.js
2	Sneider, Santiago, Mar	Empiezan controllers y routes
3	Yorman	routes/index.js y server.js
4	Todos	Cada uno registra sus rutas en index.js
5	Yorman	Integracion final y pruebas

Checklist de integracion

Quien	Archivo	Estado
Yorman	src/config/constants.js	Listo para usar
Yorman	src/middleware/auth.js	Listo para usar
Yorman	routes/index.js y server.js	Listo para usar
Yorman	controllers/auth.controller.js	Login funcional
Yorman	controllers/usuario.controller.js	Usuarios funcional
Yorman	controllers/bitacora.controller.js	Bitacora funcional
Sneider	institucion, sede, nucleo, catalogo	Entregar todo
Santiago	anio, area, asignatura, grupo, carga	Entregar todo
Martin	prematricula, matricula, indicador, calif	Entregar todo
Avila	comunicados, observador, pagos, elec	Entregar todo

8. Anexos: Codigos Base

8.1 constants.js

```
const ROLES = {
  SUPER_ADMIN: 'super_admin',
  ADMIN: 'admin', RECTOR: 'rector',
  COORDINADOR: 'coordinador', DOCENTE: 'docente',
  ESTUDIANTE: 'estudiante', ACUDIENTE: 'acudiente',
  SECRETARIA: 'secretaria',
};

const ESTADOS_USUARIO = {
  ACTIVO: 'activo', INACTIVO: 'inactivo',
  BLOQUEADO: 'bloqueado',
};

const TIPOS_DOCUMENTO = ['RC', 'TI', 'CC', 'CE', 'PAS'];
const GENEROS = ['M', 'F', 'O'];

const ESTADOS_ANIO = {
  PREMATRICULA: 'prematricula', ACTIVO: 'activo',
  CERRADO: 'cerrado',
};

const ESTADOS_PERIODO = {
  ABIERTO: 'abierto', CERRADO: 'cerrado',
};

module.exports = {
  ROLES, ESTADOS_USUARIO, TIPOS_DOCUMENTO,
  GENEROS, ESTADOS_ANIO, ESTADOS_PERIODO,
};
```

8.2 middleware/auth.js

```
const jwt = require('jsonwebtoken');
const Usuario = require('../models/Usuario');
const JWT_SECRET = process.env.JWT_SECRET
  || 'easynots_secret_key_2026';

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization
    && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return res.status(401)
      .json({ ok: false,
        message: 'No autorizado, token requerido' });
  }
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.usuario = await Usuario.findById(decoded.id)
      .select('-credenciales.passwordHash'
        + ' -credenciales.tokenRecuperacion');
    if (!req.usuario) {
      return res.status(401)
        .json({ ok: false,
          message: 'Token invalido' });
    }
    next();
  } catch (error) {
    return res.status(401)
      .json({ ok: false, message: 'Token invalido',
        error: error.message });
  }
};

const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.usuario.tipoPerfil)) {
      return res.status(403)
        .json({ ok: false,
          message: 'No tienes permisos' });
    }
  };
};
```

```
    }
    next();
  };
};

const generarToken = (usuarioid) => {
  return jwt.sign({ id: usuarioid }, JWT_SECRET,
    { expiresIn: '8h' });
};

module.exports = { protect, authorize, generarToken };
```

8.3 routes/index.js

```
const router = require('express').Router();

router.use('/auth', require('./auth.routes'));
router.use('/usuarios', require('./usuario.routes'));
router.use('/bitacora', require('./bitacora.routes'));
router.use('/instituciones',
  require('./institucion.routes'));
router.use('/sedes', require('./sede.routes'));
router.use('/nucleos',
  require('./direccionNucleo.routes'));
router.use('/solicitudes-registro',
  require('./solicitudRegistro.routes'));
router.use('/catalogos', require('./catalogo.routes'));
router.use('/anios-academicos',
  require('./anioAcademico.routes'));
router.use('/areas', require('./area.routes'));
router.use('/asignaturas',
  require('./asignatura.routes'));
router.use('/grupos', require('./grupo.routes'));
router.use('/carga-academica',
  require('./cargaAcademica.routes'));
router.use('/prematriculas',
  require('./prematricula.routes'));
router.use('/matriculas',
  require('./matricula.routes'));
router.use('/indicadores',
  require('./indicador.routes'));
router.use('/actividades',
  require('./actividad.routes'));
router.use('/calificaciones',
  require('./calificacion.routes'));
router.use('/comunicados',
  require('./comunicados.routes'));
router.use('/observador',
  require('./observador.routes'));
router.use('/excusas', require('./excusas.routes'));
router.use('/conceptos-contables',
  require('./conceptosContables.routes'));
router.use('/pagos', require('./pagos.routes'));
router.use('/elecciones',
  require('./elecciones.routes'));
router.use('/eventos-electorales',
  require('./eventoElectoral.routes'));
router.use('/votos', require('./voto.routes'));

module.exports = router;
```

8.4 server.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const mongoose = require('mongoose');

const app = express();

app.use(cors());
app.use(helmet());
app.use(express.json());
app.use(rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 200,
```

```
});  
  
app.use('/api',  
  require('./src/routes/index'));  
  
const PORT = process.env.PORT || 3000;  
  
mongoose.connect(process.env.MONGODB_URI)  
  .then(() => {  
    console.log('[OK] MongoDB conectado');  
    app.listen(PORT, () =>  
      console.log('[OK] Servidor en puerto', PORT));  
  })  
  .catch(err => {  
    console.error('[ERROR] MongoDB:', err.message);  
    process.exit(1);  
  });
```